

가천대학교 스마트시티학과 – 가상공간과 메타버스

Lecture 18 – Validation and Final Integration

Chris Ohk
utilForever@gmail.com

학습 목표

- 코드가 규칙대로 계산하는지 작은 예제로 확인한다.
- 같은 도시를 여러 번 실행하고 결과의 차이를 설명한다.
- 기능을 연결한 뒤 오류와 사용하기 불편한 부분을 찾는다.
- 다른 PC에서도 실행되는 최종본과 시연 순서를 준비한다.

우리가 만든 도시 점검하기

- 기능이 따로 잘 돌아가도, 함께 실행하면 문제가 생길 수 있다.

이미 만든 기능	이제 확인할 것
도로와 이동	도로를 지우면 이동 경로도 바뀌는가?
병원과 대기열	진료한 사람과 남은 사람의 수가 맞는가?
지표와 화면	계산한 값이 그래프와 저장 파일에도 같게 남는가?

병원 결과에서 확인할 것

- 대기 인원 14명이라는 결과만 보고 "맞다"고 판단할 수는 없다.

확인할 질문	병원 예제
계산이 맞는가?	10명이 기다리던 중 10명이 더 왔다. 6명을 진료하면 14명이 남는가?
가정이 적절한가?	실제 병원에도 매시간 정확히 10명씩 오는가?
다른 입력에서도 비슷한가?	방문 인원이 달라도 병원을 늘린 효과가 있는가?

코드 검증과 모델 타당성

- **코드 검증Code Verification**이란 규칙대로 계산했는지 확인하는 것이고, **모델 타당성 평가Model Validation**란 목적에 맞는지 살피는 것이다.

구분	코드 검증	모델 타당성
무엇을 확인하나?	코드의 계산과 동작	모형의 가정과 결과
병원 예제	남은 인원을 14명으로 계산하는가?	방문 인원과 진료 규칙이 적절한가?
확인 방법	손 계산과 코드 결과 비교	실제 자료나 알려진 특성과 비교

사람 수가 맞아야 한다

- 진료한 사람과 남은 사람을 합하면, 전체 인원과 같아야 한다.
  - 기다리던 사람 + 새로 온 사람 = 진료한 사람 + 남은 사람
- 인원은 0명보다 적어질 수 없고,

  진료한 인원은 기다린 전체 인원을 넘을 수 없다.
  - 이번 예제에서는 기다리다 떠나는 사람이 없다고 가정한다.

결과만 보고 판단하지 말자

- 2명이 기다리고 3명이 새로 왔다면, 최대 5명만 진료할 수 있다.

확인할 값	잘못된 코드	올바른 코드
진료할 수 있는 인원	10명	10명
실제로 진료한 인원	10명으로 기록	$\min(2+3, 10) = 5$명
남은 대기	음수를 0으로 바꿈	$2 + 3 - 5 = 0$명
문제	오지 않은 5명도 진료한 셈	진료 5명 + 대기 0명

극단적인 입력 테스트

- 아무도 오지 않거나, 진료를 전혀 못 하거나,
  사람이 아주 많이 오면 어떻게 될까?

기다림 / 새로 옴 / 진료 용량	진료한 사람	남은 사람
0명 / 0명 / 8명	0명	0명
2명 / 10명 / 0명	0명	12명
2명 / 10명 / 100명	12명	0명
0명 / 1,000명 / 8명	8명	992명

0과 "알 수 없음"의 차이

- 도로가 끊겼을 때 이동시간을 0분으로 표시하면 잘못 이해하게 된다.

상황	표시하거나 처리할 방법
기다리는 사람이 없음	대기 0명
병원까지 갈 길이 없음	경로 없음
아직 기록한 값이 없음	기록 없음
진료 용량에 −3을 입력	입력을 거부하고 이유를 안내

모형의 가정 확인하기

- 대기열 원리를 설명할지, 실제 병원의 인력을 정할지에 따라 필요한 자료가 다르다.

확인할 것	쉽게 생각하면
입력 자료	하루 방문 인원을 시간당 인원으로 잘못 넣지 않았는가?
결과의 변화	방문 인원이 진료 용량보다 많으면 대기가 늘어나는가?
사용할 범위	평상시 자료만으로 재난 때의 결과까지 설명하고 있지 않은가?

다른 자료로 다시 확인하기

- 월요일 자료에 맞춘 모형이 화요일 자료도 설명하는지 확인한다.

단계	설명용 예
① 값을 조정한다	월요일 방문 기록을 보고 방문 인원 설정을 조정한다.
② 다른 자료와 비교한다	조정할 때 쓰지 않은 화요일 기록과 비교한다.
③ 차이가 나는 곳을 찾는다	특히 사람이 몰리는 시간대에서 차이가 큰지 살펴본다.

병원 예제의 범위

- 예제는 방문 인원과 진료 용량에 따라 대기가 달라지는 모습을 설명한다.
- 환자의 중증도, 진료 종류, 기다리다 떠나는 행동은 넣지 않았다.
- 가상으로 만든 숫자이므로 실제 병원의 대기를 예측한 결과는 아니다.

난수와 난수 시드

- 방문 인원을 매번 다르게 정할 때 **난수Pesudo-random Number**를 사용한다.
  - 예 : 매시간 6명부터 14명 사이에서 하나를 뽑는다.
- 같은 숫자 순서를 다시 만들기 위한 시작값을 **시드Seed**값이라고 한다.
  - 같은 환경에서 시드를 다시 같게 설정하고 같은 순서로 뽑으면 같은 숫자가 나온다.

같은 방문 인원 다시 만들기

- 시드를 101로 다시 설정하면 같은 6시간 분량의 방문 인원을 만들 수 있다.

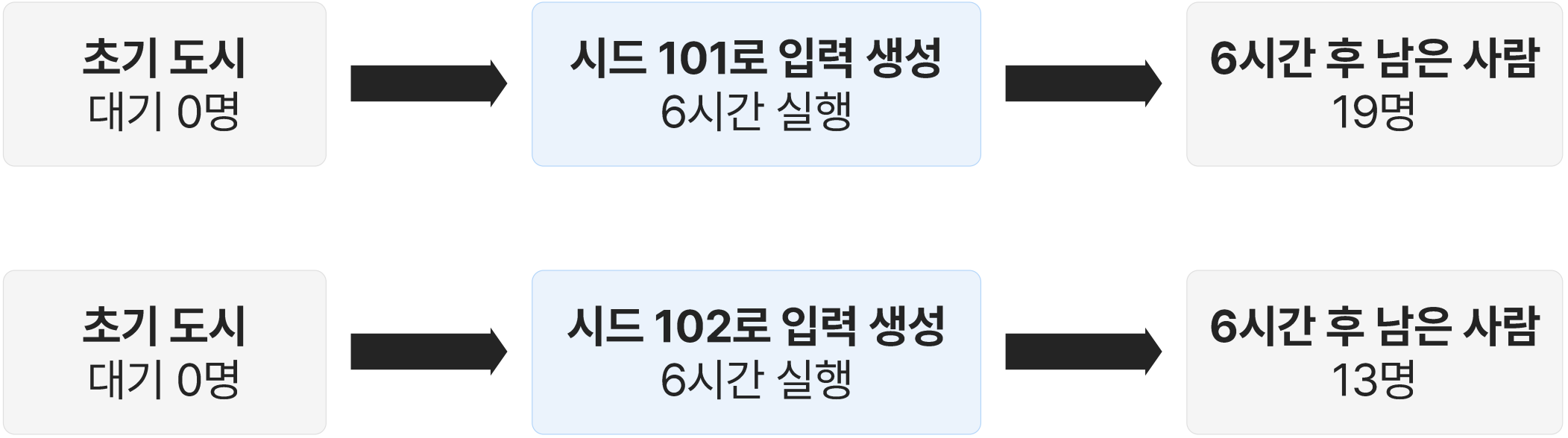
실행	시드	1~6시간째 방문 인원
처음 실행	101	9 / 14 / 11 / 13 / 6 / 14명
다시 실행	101	9 / 14 / 11 / 13 / 6 / 14명
시드를 바꿔 실행	102	8 / 11 / 8 / 14 / 8 / 12명

같은 결과를 다시 얻으려면

- 시드뿐 아니라 시작 도시, 정책 값, 실행 시간을 함께 기록한다.
- 같은 프로그램과 같은 Python 버전에서 다시 실행한다.
- 비교할 때는 방문 인원 목록을 저장해 두고 똑같이 사용한다.
  - 코드가 난수를 뽑는 순서를 바꾸면, 같은 시드에서도 입력이 달라질 수 있다.

반복 실험

- 매번 같은 초기 도시로 돌아간 뒤, 다른 방문 인원으로 다시 실행한다.



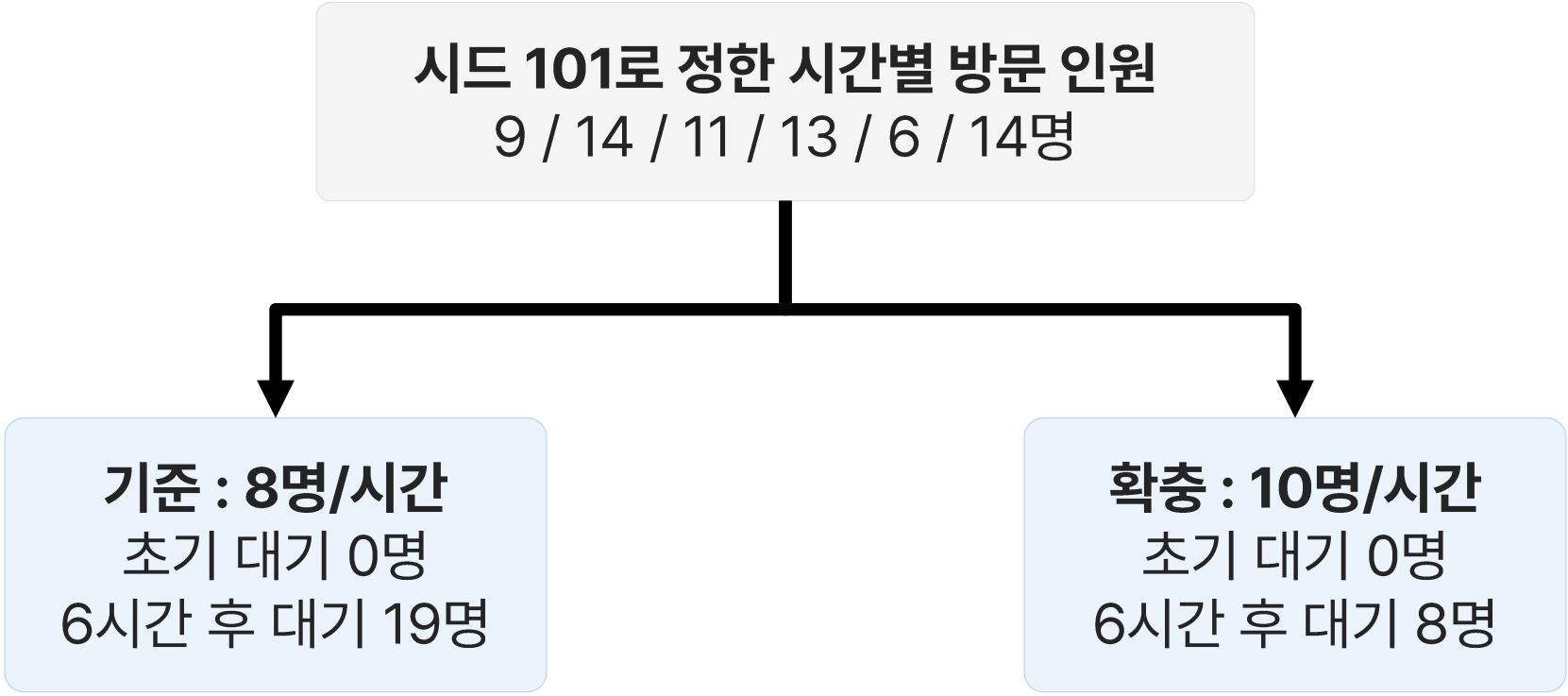
기준 시나리오와 비교 시나리오

- 실행 조건을 묶은 것을 **시나리오Scenario**라고 한다.
  먼저 기준을 정한 뒤 바꿀 조건을 고른다.

실행 조건	기준 시나리오	비교 시나리오
시작 대기 / 실행 시간	0명 / 6시간	0명 / 6시간
시간당 진료 가능 인원	8명	10명
방문 인원	매시간 6~14명	기준과 같은 방문 인원
도로 / 전력 / 급수	도로 고정 / 공급 충분	기준과 같은 조건

같은 방문 인원으로 비교하기

- 같은 6시간 분량의 방문 인원을 두 병원 모형에 넣는다.



5번 실행한 결과

- 모두 6시간 후의 대기 인원이다. 병원을 늘린 뒤에도 결과는 매번 다르다.

시드	기준 8명/시간	확충 10명/시간
101	19명	8명
102	13명	4명
103	13명	5명
104	7명	0명
105	11명	5명

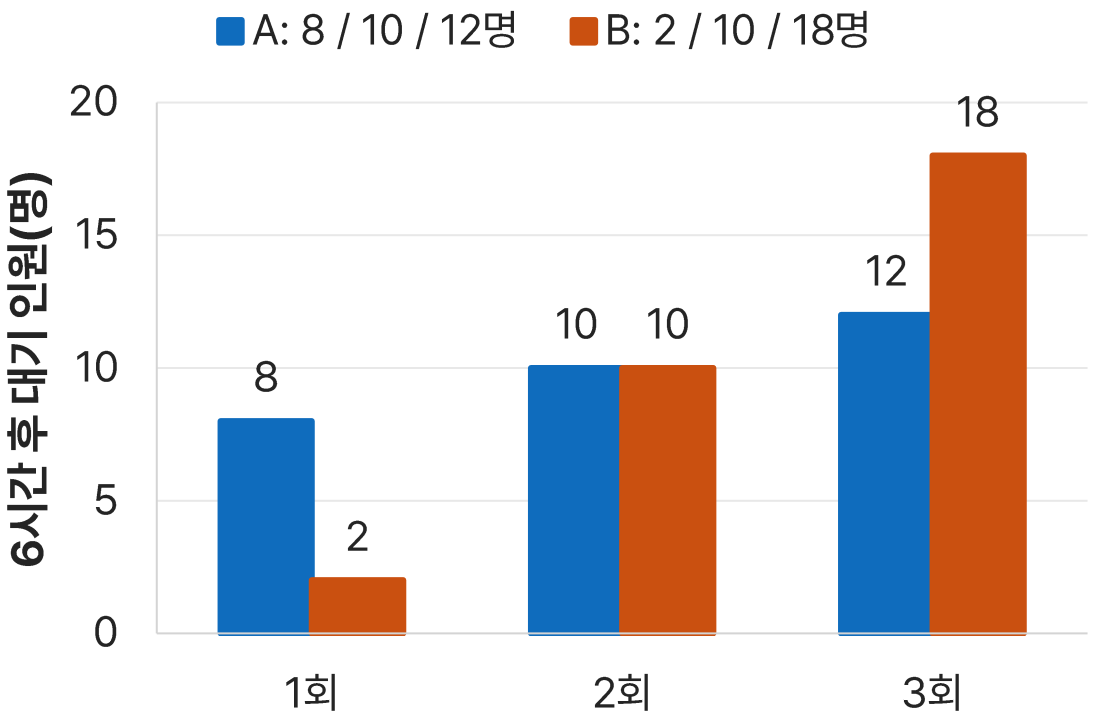
대기가 얼마나 줄었을까?

- 줄어든 인원 = 기준의 대기 인원 – 확충 후 대기 인원

시드	계산	줄어든 인원
101	$19 - 8$	11명
102	$13 - 4$	9명
103	$13 - 5$	8명
104	$7 - 0$	7명
105	$11 - 5$	6명

평균이 같아도 결과는 다르다

- A와 B의 평균은 모두 10명이다.
  그러나 B는 2명부터 18명까지 크게 달라진다.



변동성과 표준편차

- **표준편차Standard Deviation**란 결과가 평균 주변에 얼마나 흩어져 있는지 나타내는 값이다.

결과	평균	표준편차	가장 작은 값~큰 값
A : 8 / 10 / 12명	10명	2명	8~12명
B : 2 / 10 / 18명	10명	8명	2~18명

30회 실험 결과 읽기

- 병원을 늘리면 평균 대기는 줄어들지만, 여전히 대기가 있을 때가 있다.

지표	기준 8명/시간	확충 10명/시간
실험 횟수	30회	30회
평균	12.60명	3.93명
표본 표준편차	5.47명	3.18명
최솟값~최댓값	2~22명	0~10명

몇 번 반복해야 할까?

- 먼저 몇 번 실행해 초기화와 기록이 제대로 되는지 확인한다.
- 횟수를 늘렸을 때 평균과 정책의 차이가 크게 바뀌는지 본다.
- 결과가 들쭉날쭉할수록 더 많은 실행이 필요할 수 있다.
- 30회는 예시일 뿐이며, 모형에 따라 더 많이 실행해봐야 한다.

실험 기록 남기기

- 나중에 같은 실험을 다시 할 수 있도록 조건과 결과를 함께 저장한다.

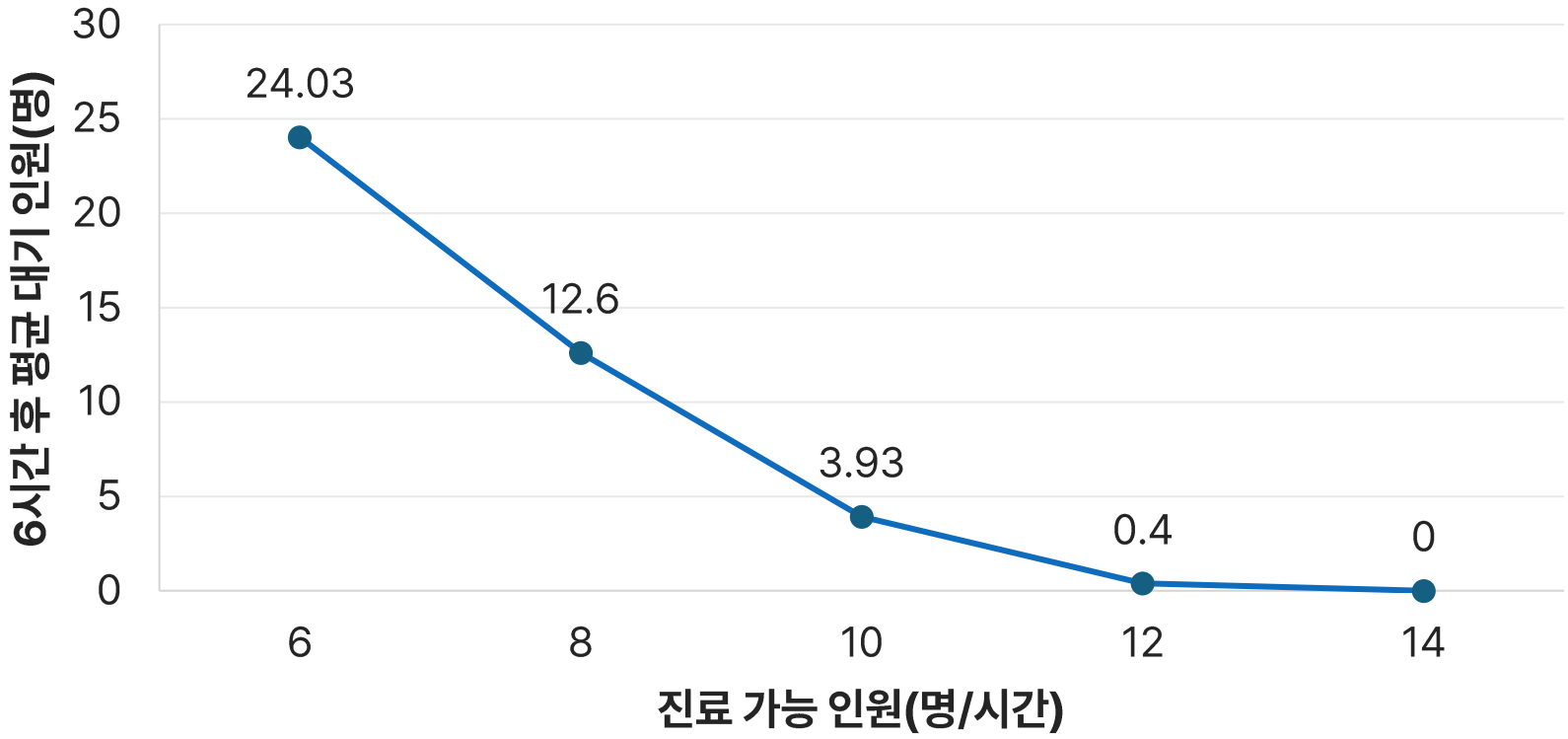
기록할 것	예제 기록
실험 번호와 시드	1회 / 시드 101
시작 상태와 정책	대기 0명 / 시간당 진료 8명
실행 시간과 입력	6시간 / 시간별 방문 인원 목록
결과와 프로그램 버전	6시간 후 대기 19명 / 사용한 코드 버전

민감도 분석

- **민감도 분석Sensitivity Analysis**이란 설정값을 바꿀 때 결과가 얼마나 달라지는지 살펴보는 방법이다.
  - 예 : 시간당 진료 가능 인원을 6명, 8명, 10명으로 바꾼다.
- 먼저 비교할 값의 범위와 결과 지표를 정한다.
- 방문 인원과 다른 조건은 같게 두고, 한 항목부터 바꾸어 본다.

병원 용량을 바꾸면

- 30회 실험과 같은 방문 인원을 사용하고, 시간당 진료 가능 인원만 바꿨다.



병원만 늘려도 될까?

- 하지만 물이 부족하면 병원을 늘려도 더 많이 진료할 수 없다.

6시간 후 평균 대기	급수로 진료 가능한 인원 8명/시간	급수로 진료 가능한 인원 10명/시간
병원 용량 : 8명/시간	12.60명	12.60명
병원 용량 : 10명/시간	12.60명	3.93명

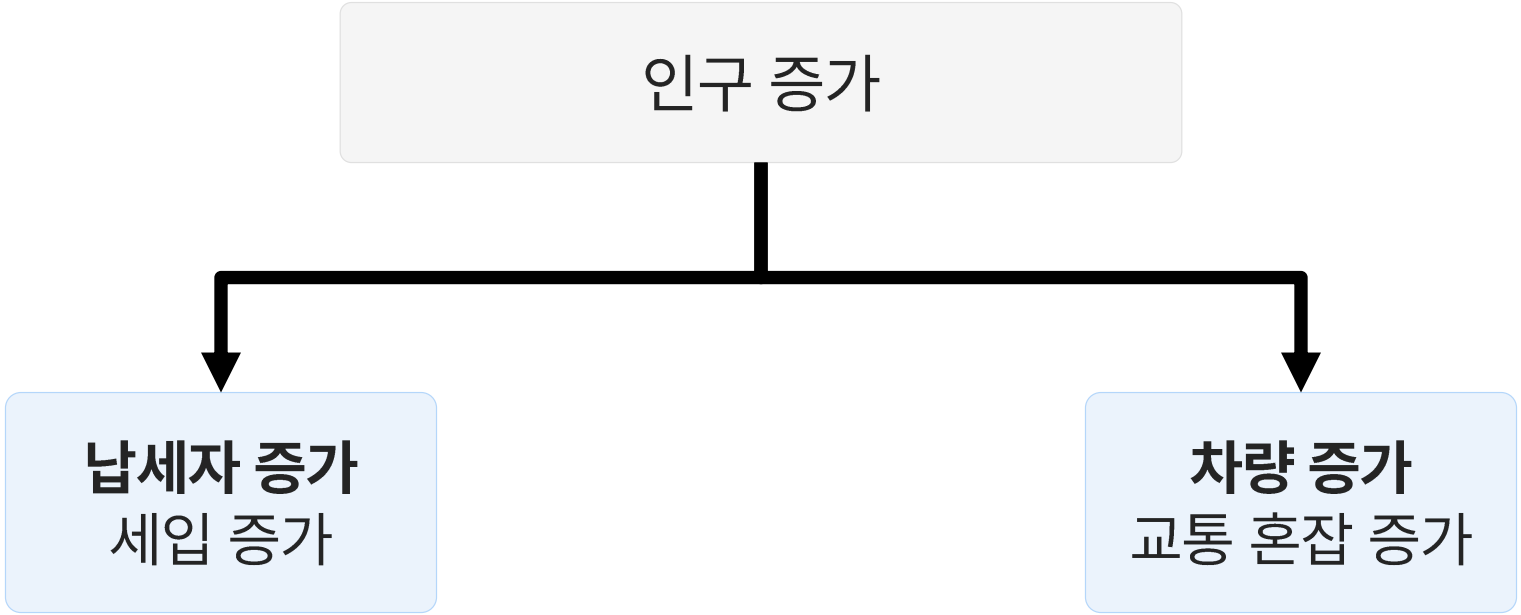
대기 인원만 보면 안되는 이유

- 대기가 줄어도 비용이 너무 늘거나, 일부 지역만 편해질 수 있다.

함께 볼 지표	확인할 질문
대기 인원	병원 이용이 더 편해졌는가?
예산과 유지비	늘어난 인력과 시설 비용을 감당할 수 있는가?
접근시간	멀리 사는 시민도 병원에 가기 쉬워졌는가?
지역별 결과	한 지역만 좋아진 것은 아닌가?

상관관계와 인과관계

- **상관관계Correlation**란 두 값이 함께 변하는 관계를 말한다.
- **인과관계Causation**란 한쪽의 변화가 다른 쪽을 바꾸는 관계를 말한다.
- 세입과 혼잡이 함께 늘어도, 세입이 혼잡을 일으켰다고 단정할 수는 없다.

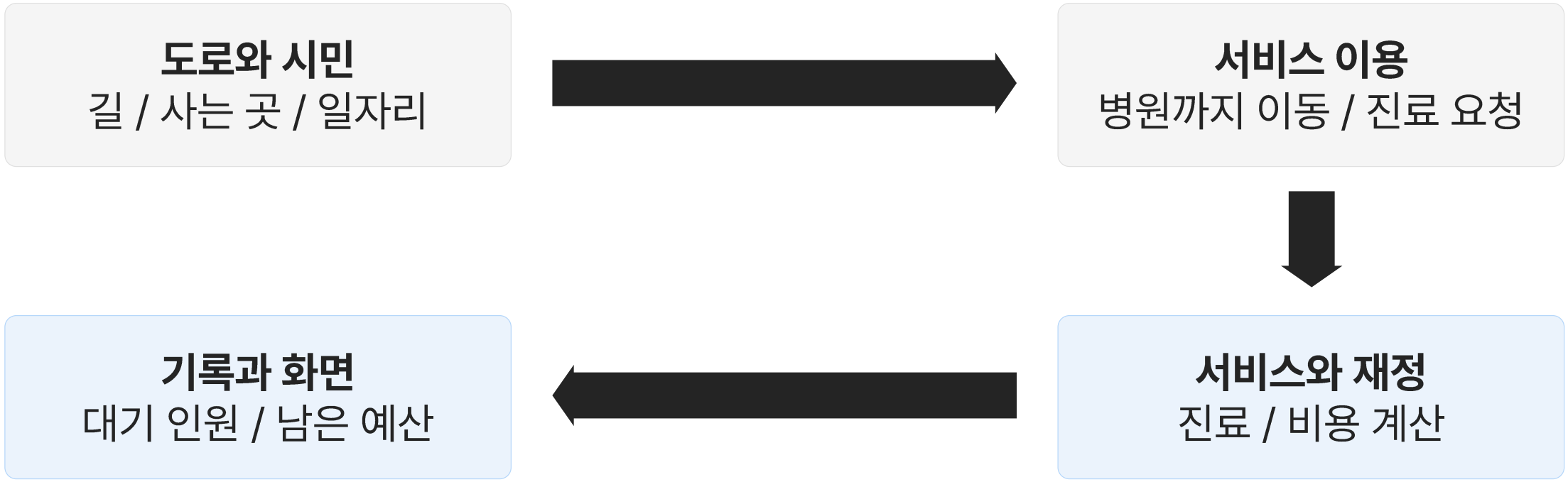


모형 안에서 정책 효과 확인하기

- 병원 용량을 바꾸고, 나머지 조건은 같게 두어 비교한다.
- 차이가 생기면 먼저 코드와 기록이 맞는지 확인한다.
- 확인한 차이는 이 모형의 규칙과 가정에서 나온 결과일 뿐이다.
- 실제 도시에서도 같은 효과가 날지는 실제 자료로 더 확인해야 한다.

기능 통합

- 따로 만든 기능들이 도시 정보를 주고받도록 연결하는 일을 **기능 통합Function Integration**이라고 한다.



처음부터 끝까지 실행해 보기

- 기능을 통합한 후에는 화면에 값들이 모두 맞게 바뀌는지 확인한다.

해 볼 조작	기대하는 결과
도로를 지운다	이동 경로와 접근시간이 다시 계산된다.
정책을 바꾸고 진행한다	정해진 적용 시점부터 새 정책으로 계산한다.
과거 그래프를 누른다	그때의 기록을 보여 주되 현재 도시는 그대로 둔다.
저장한 도시를 다시 연다	같은 상태에서 다음 실행을 이어 갈 수 있다.

통합한 뒤 자주 생기는 오류

- 화면만 보지 말고, 어떤 값이 언제 바뀌었는지도 확인한다.

보이는 문제	먼저 확인할 부분
도로를 지워도 옛길로 이동한다	저장해 둔 경로를 다시 계산했는가?
과거 그래프가 현재 값으로 바뀐다	그때의 값을 따로 복사해 저장했는가?
탭을 바꾸면 대기가 늘어난다	화면을 그릴 때 계산까지 다시 하는가?
새 실험에 지난 이벤트가 남는다	도시 상태와 기록을 모두 초기화했는가?

오류가 생긴 순서 기록하기

- 단순히 “안 된다”고 말하지 말고, 무엇을 했을 때 무엇이 달랐는지 적는다.

기록할 것	예시
시작 조건	기본 도시 파일 / 시드 101 / 사용한 코드 버전
조작 순서	도시 열기 → 도로 삭제 → 이동 경로 보기
기대한 결과	삭제한 도로를 피해서 새 경로가 나옴
실제로 나온 결과	삭제한 도로를 지나는 예전 경로가 남음

오류를 고친 뒤 다시 확인하기

- 오류가 나타나는 가장 작은 도시와 짧은 실행 순서를 만든다.
- 처음 잘못된 값이 생긴 부분을 찾아 고친다.
- 같은 입력을 다시 넣어 예상한 결과가 나오는지 확인한다.
- 그 값을 사용하는 다른 기능과 이전에 잘되던 예제도 다시 실행한다.

먼저 고칠 오류 정하기

- 실행을 막거나 결과를 잘못 해석하게 만드는 문제부터 고친다.

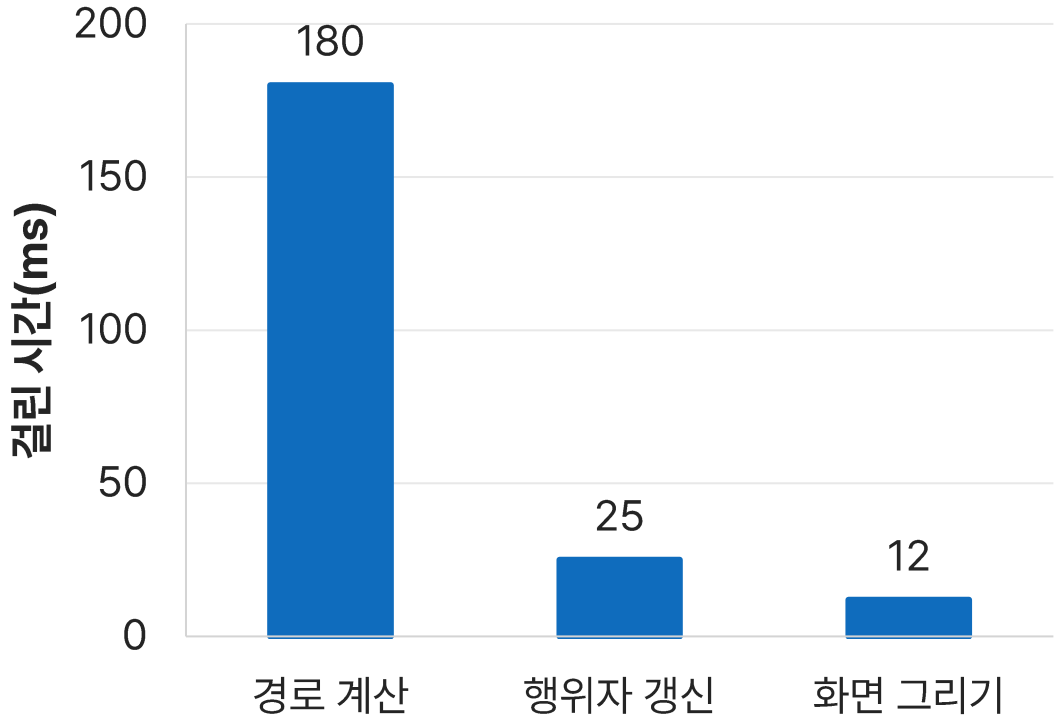
우선순위	예시
높음	실행 불가 / 저장 파일 손상 / 잘못된 계산
중간	너무 긴 대기 / 잘못된 경고 / 어려운 조작
낮음	줄바꿈 / 정렬 / 표현의 불일치

성능 점검

- **성능 점검Performance Check**이란 작업을 실행하는 데
  얼마나 시간이 걸리는지 확인하는 것이다.
- 경로 계산, 시민 상태 변경, 화면 그리기를 나누어 시간을 잰다.
- 도시 크기와 시민 수를 적고, 같은 PC에서 여러 번 측정한다.
  - 처음 파일을 읽는 시간과 실행 중 계산하는 시간도 구분한다.

어느 작업이 오래 걸릴까?

- 예제를 보면, 경로 계산이 오래 걸리므로 이 부분부터 살펴본다.



빨라진 뒤에도 결과 확인하기

- 실행 시간이 줄었더라도 계산 결과가 달라지면 원인을 확인해야 한다.

맞출 조건	확인 방법
같은 입력	같은 도시 / 시민 수 / 방문 인원 / 실행 시간
같은 측정 환경	같은 PC에서 같은 구간을 여러 번 측정
같은 계산 결과	대기 인원, 비용, 저장된 기록을 비교
바뀐 입력의 처리	도로를 고치면 저장해 둔 경로도 다시 계산

친구에게 사용해 보게 하기

- 프로그램을 처음 쓰는 사람이 실행해서 사용하게 한다.
- “이 버튼을 누르세요” 대신 해야 할 일을 알려 준다.
  - 도시 열기 / 병원 용량 바꾸기 / 결과 비교 / 저장하기
- 제작자는 옆에서 지켜보며 헷갈렸던 부분과 질문을 기록한다.

사용성 테스트 기록

- 무엇을 하려다 어디에서 헷갈렸는지 적으면 수정할 부분이 보인다.

하려던 일	헷갈린 점	고칠 부분
정책 변경	언제 적용되는지 모름	적용 시점과 현재 값 표시
그래프 확인	현재 값과 과거 값을 혼동	선택한 시점 표시
상태 저장	저장됐는지 모름	파일명과 완료 메시지 표시
지표 비교	색의 뜻을 모름	숫자, 단위, 범례 표시

다른 PC에서 실행해 보기

- 내 PC에서 동작한다고 해서 발표용 PC에서도 된다는 보장은 없다.

확인할 것	직접 해 볼 일
설치 환경	사용한 Python과 패키지 버전으로 설치
화면	Tkinter 실행 / 글자 크기 / 화면 잘림 확인
도시 파일	새 폴더에 풀어 예제 도시 열기와 저장하기
인터넷 연결	연결이 없을 때 필요한 시연이 가능한지 확인

실행할 파일 묶기

- 프로그램뿐 아니라 예제 도시와 실행 설명서도 함께 준비한다.

파일	들어갈 내용
프로그램	실행 파일과 필요한 Python 코드
예제 도시	시작 도시 / 정책 설정 / 필요한 입력 자료
실행 설명서	설치 방법 / 실행 명령 / 조작 방법
실험 결과	실행 조건 / 시드 / 회차별 결과 / 요약 표

발표용 예제 도시 준비

- 짧은 시간 안에 도시의 특징과 정책의 차이가 보이는 예제를 준비한다.

예제	보여 줄 내용
기본 도시	어떤 문제가 있고, 현재 지표가 얼마인지
정책을 바꾼 도시	같은 조건에서 정책 하나를 바꾸면 어떤 차이가 나는지
예외 상황	도로가 끊기거나 물이 부족할 때 어떻게 처리하는지

시연 순서 작성

- 아래는 15분 시연의 예다. 실제 발표 시간은 별도 안내에 맞춘다.

시간	시연 내용	설명할 핵심
0:00~0:30	주제와 초기 도시 소개	해결하려는 문제
0:30~1:30	기준 시나리오 실행	초기 상태 / 지표
1:30~3:00	정책 하나를 바꾸어 실행	같은 입력 / 달라진 결과
3:00~4:00	반복 실험 표와 한계 설명	평균 / 변동성 / 적용 범위
4:00~5:00	경계 사례와 저장 상태 확인	오류 처리 / 재현 방법

시연 중 문제가 생기면

- 저장한 예제 도시로 돌아가 처음 상태에서 다시 실행한다.
- 같은 오류가 반복되면 현장에서 무리하게 코드를 고치지 않는다.
- 저장한 결과나 영상을 사용하면 미리 만든 자료임을 설명한다.
- 동작한 부분과 아직 해결하지 못한 부분을 구분해 답한다.

발표 리허설

- 프로그램을 닫아 둔 상태에서 시작해 실제 순서대로 시연한다.
- 설명, 마우스 조작, 계산을 기다리는 시간을 모두 잰다.
- 동료에게 글자가 잘 보이고 설명이 이해되는지 확인받는다.
- 연습 중 코드를 바꿨다면 바뀐 부분을 다시 실행한다.

코드 동결

- 확인한 버전을 최종본으로 정하고 새 기능 추가를 멈추는 것을 **코드 동결Code Freeze**이라고 한다.

시점	할 일
동결 전	필수 기능과 오류를 정리하고 리허설을 마친다.
동결 후	코드, 예제 도시, 설정, 실험 결과를 함께 보관한다.
중요한 오류 발견	고친 뒤 관련 검사와 시연을 다시 하고 최종본을 갱신한다.

최종 준비 상태 확인

- 다시 볼 수 있는 결과와 파일을 남긴 후 살펴 보며 점검한다.

점검 항목	준비할 자료
계산과 가정	작은 테스트의 답 / 모형의 가정과 한계
반복 실험	실행 조건 / 회차별 결과 / 평균과 변동성
프로그램	오류 수정 기록 / 사용성 확인 / 실행 설명서
발표	예제 도시 / 시연 순서 / 확인한 최종 버전

확인 문제

- 도로를 지웠는데 시민이 여전히 옛길로 이동한다. 무엇부터 확인할까?
- 내 PC에서 잘 실행되면 발표용 PC에서는 확인하지 않아도 될까?
- 코드 동결 뒤 대기 인원 계산이 틀렸다는 것을 알았다. 어떻게 해야 할까?

문제 해설

- 도로 변경이 경로 계산에 전달됐는지 보고,
  저장해 둔 옛 경로를 다시 계산한다.
- 발표용 PC에서도 설치, 파일 열기, 화면 표시를 직접 확인한다.
- 잘못된 계산은 고치고, 관련 기능과 시연을 다시 실행한 뒤
  최종본을 갱신한다.

정리

- 작은 입력으로 계산을 확인하고, 모형의 가정이 목적에 맞는지 살펴본다.
- 같은 도시에서 여러 번 실험하고 평균과 결과의 차이를 함께 읽는다.
- 기능을 연결한 뒤 오류, 속도, 사용하기 어려운 부분을 확인한다.
- 다른 PC에서 실행해 보고, 예제 도시와 시연 순서를 준비한 뒤 최종본을 정한다.

감사합니다.

utilForever@gmail.com

https://github.com/utilForever

Facebook, Twitter, Instagram, LinkedIn: utilForever